

Laporan Tugas Besar Milestone 3

**IF2224 Teori Bahasa Formal dan Otomata
Semantic Analysis
Semester II 2025/2026**



Disusun oleh:

Fauzan Mohamad Abdul Ghani	13524113
Safira Berlianti	13524128
Eduard Daniel Ariajaya	13524129
Muhammad Daffa Arrizki Yanma	13524133
Reysha Syafitri Mulya Ramadhan	13524137

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026**

Daftar Isi

BAB I Teori Singkat.....	4
1.1. Semantic Analysis.....	4
1.2. Abstract Syntax Tree (AST).....	4
1.3. Decorated Abstract Syntax Tree (Decorated AST).....	6
1.4. Attributed Grammar.....	6
1.5. Symbol Table.....	7
1.6. Visitor Function.....	7
BAB II Perancangan & Implementasi.....	8
2.1. Perancangan Program.....	8
2.1.1. Teknologi yang Digunakan.....	8
2.1.2. Struktur Program.....	8
2.1.3. Alur Kerja Program.....	8
2.1.4. Justifikasi Pilihan Implementasi.....	9
2.2. Implementasi.....	12
2.2.1. ASTNode.hpp.....	12
2.2.2. SymbolTable.hpp.....	14
2.2.3. SemanticAnalyzer.hpp/.cpp.....	17
2.2.4. ASTDecoratedPrinter.hpp/.cpp.....	30
2.3. Semantic Action.....	36
2.4. Predefined Identifier.....	39
2.5. Type Compatibility Rule.....	40
2.5.1. Type Compatibility (untuk operasi biner, relasional, aritmatika).....	40
2.5.2. Assignment Compatibility (untuk statement assignment).....	40
2.5.3. Aturan Tambahan (diimplementasikan di berbagai visitor).....	40
BAB III Pengujian.....	42
3.1. Test Case 1.....	42
3.2. Test Case 2.....	44
3.3. Test Case 3.....	47
3.4. Test Case 4.....	49
3.5. Test Case 5.....	51
3.6. Test Case 6 (Error Handling).....	54
3.7. Test Case 7 (Error Handling).....	56
3.8. Test Case 8 (Error Handling).....	58
3.9. Test Case 9 (Error Handling).....	61
3.10. Test Case 10 (Error Handling).....	63
BAB IV Penutup.....	67
4.1. Kesimpulan.....	67

4.2. Saran.....	67
LAMPIRAN.....	69
REFERENSI.....	71

BAB I

Teori Singkat

1.1. Semantic Analysis

Semantic Analysis merupakan tahap lanjutan setelah syntax analysis dalam proses kompilasi. Pada tahap ini, compiler tidak hanya memeriksa apakah program sesuai dengan grammar bahasa, tetapi juga memastikan bahwa program memiliki makna yang valid secara semantik.

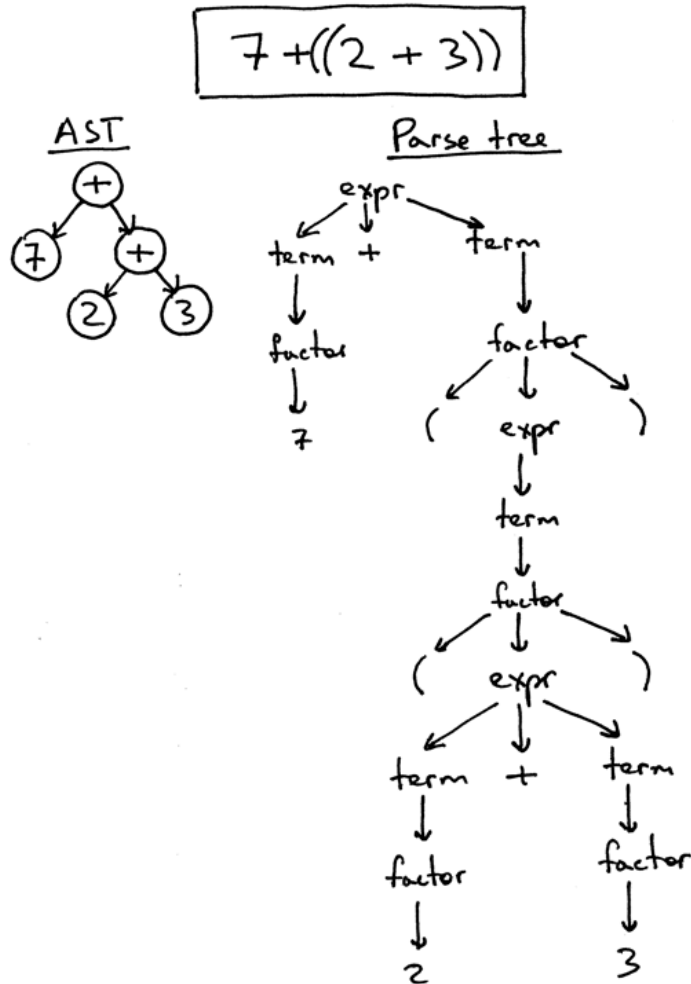
Semantic analyzer bekerja menggunakan hasil parsing berupa Abstract Syntax Tree (AST) dan symbol table untuk melakukan berbagai pengecekan, seperti:

- ***Type Checking (Pengecekan Tipe)***
Memastikan operator dan operand memiliki tipe yang kompatibel.
Contoh: tidak menjumlahkan string dengan integer, perbandingan char terhadap string.
- ***Symbol Table Management (Pengecekan Deklarasi)***
Memastikan setiap identifier (variabel/fungsi) telah dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang dalam cakupan yang sama.
- ***Scope Resolution (Pengecekan Lingkup)***
Menentukan validitas akses variabel berdasarkan hierarki blok kode.
Contoh: scope global vs scope local.
- ***Control Flow Validation (Pengecekan Alur Kontrol)***
Memastikan struktur alur kontrol yang logis.
Contoh: instruksi break/continue hanya dalam loop, fungsi dipastikan memiliki return pada semua cabang kontrol, warning untuk unreachable code.

1.2. Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) merupakan representasi struktur program dalam bentuk pohon yang lebih sederhana dibandingkan parse tree. AST hanya menyimpan informasi penting yang berkaitan dengan struktur sintaks dan semantik program, tanpa menyertakan detail grammar yang tidak diperlukan.

Setiap node pada AST merepresentasikan konstruksi program tertentu, seperti deklarasi variabel, operasi aritmatika, assignment, maupun statement kontrol. Struktur AST mempermudah compiler dalam melakukan traversal dan analisis program pada tahap semantic analysis maupun code generation.

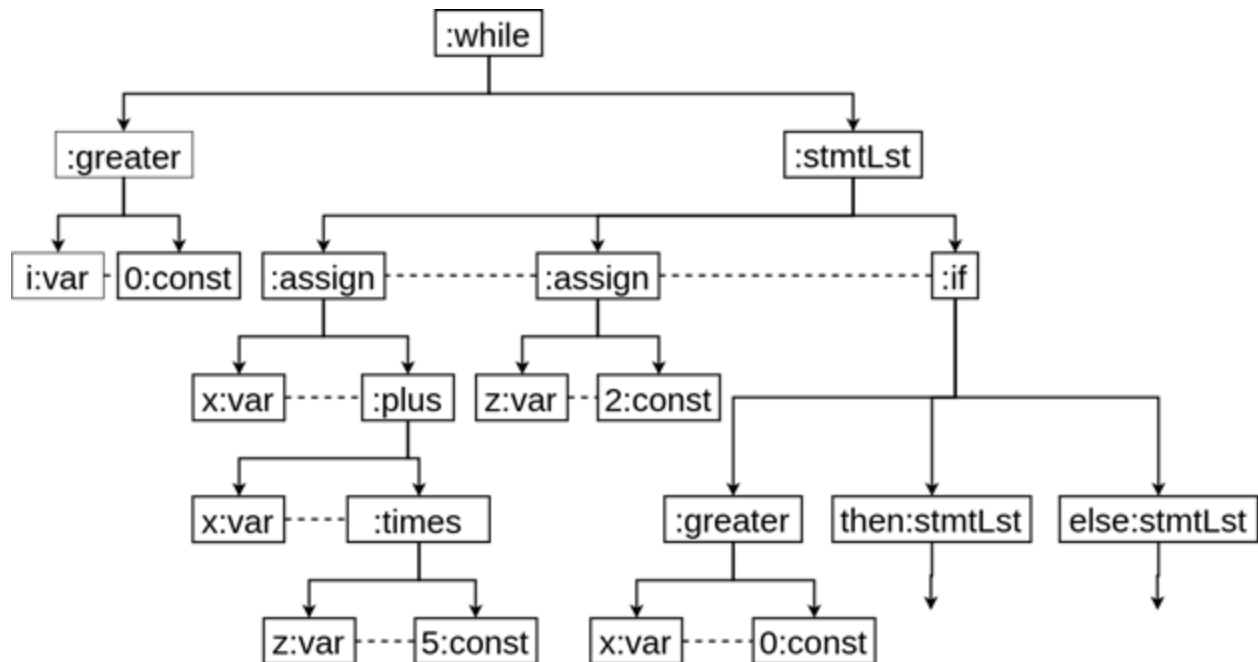


Gambar Parse Tree dan Abstract Syntax Tree (AST) untuk ekspresi matematika
 (Sumber: <https://ruslanspivak.com/lsbasi-part7/>)

Sebagai ilustrasi, untuk potongan kode berikut.

```
/* TIDAK DIPASTIKAN SESUAI BAHASA ARION */
while (i > 0) {
    x = x + z * 5;
    z = 2;
    if (x > 0) then { ... } else { ... }
}
```

Kode tersebut akan menghasilkan Abstract Syntax Tree:



Gambar Contoh Abstract Syntax Tree (AST)

(Sumber: <https://nus-cs3203.github.io>)

1.3. Decorated Abstract Syntax Tree (Decorated AST)

Setelah semantic analysis dilakukan, AST akan berkembang menjadi Decorated AST atau Attributed AST. Pada struktur ini, setiap node AST diberikan atribut tambahan berupa informasi semantik, seperti tipe data hasil evaluasi, referensi ke symbol table, maupun informasi scope. Atribut tersebut digunakan compiler untuk membantu proses validasi semantik dan mendukung tahap lanjutan seperti intermediate code generation.

Decorated AST memungkinkan compiler menyimpan hasil inferensi semantik secara langsung pada struktur pohon program sehingga proses analisis berikutnya dapat dilakukan dengan lebih efisien.

1.4. Attributed Grammar

Attributed Grammar merupakan pengembangan dari Context-Free Grammar (CFG) dengan menambahkan atribut dan aturan semantik pada setiap simbol grammar. Atribut digunakan untuk menyimpan informasi tambahan, seperti tipe data, nilai ekspresi, maupun referensi identifier.

Secara umum, atribut dibagi menjadi dua, yaitu Synthesized Attribute (atribut yang dihitung dari child node menuju parent node) dan Inherited Attribute (atribut yang diwariskan dari parent node ke child node). Dalam semantic analysis, attributed grammar digunakan untuk

membantu proses perhitungan tipe data, pengecekan deklarasi, serta validasi aturan semantik lainnya selama traversal AST.

1.5. Symbol Table

Symbol Table merupakan struktur data yang digunakan compiler untuk menyimpan informasi mengenai identifier yang muncul dalam program, seperti nama variabel, tipe data, scope, parameter fungsi, maupun atribut tambahan lainnya. Symbol table menggunakan struktur data stack.

Semantic analyzer menggunakan symbol table untuk memastikan identifier telah dideklarasikan sebelum digunakan, mendeteksi deklarasi ganda, melakukan pencarian identifier berdasarkan scope, serta membantu proses type checking.

- **Identifier Table (tab)**

Identifier table digunakan untuk menyimpan informasi utama setiap identifier dalam program, seperti nama, tipe data, jenis objek, level scope, dan alamat referensi tertentu. Tabel ini digunakan untuk proses pencarian identifier selama semantic analysis.

- **Block Table (btab)**

Block table digunakan untuk menyimpan informasi mengenai blok program, seperti prosedur, fungsi, maupun record. Informasi yang disimpan dapat berupa parameter, ukuran variabel lokal, dan hubungan antar scope.

- **Array Table (atab)**

Block table digunakan untuk menyimpan informasi mengenai blok program, seperti prosedur, fungsi, maupun record. Informasi yang disimpan dapat berupa parameter, ukuran variabel lokal, dan hubungan antar scope.

1.6. Visitor Function

Traversal AST pada semantic analysis umumnya dilakukan menggunakan pendekatan Visitor Function atau Visitor Pattern. Pada pendekatan ini, setiap jenis node AST memiliki fungsi khusus untuk melakukan analisis semantik terhadap node tersebut. Sebagai contoh fungsi untuk assignment statement, fungsi untuk operasi biner, fungsi untuk deklarasi variabel, maupun fungsi untuk ekspresi. Setiap fungsi visitor dapat melakukan traversal ke child node, mengakses atau memperbarui symbol table, melakukan pengecekan tipe data, menambahkan atribut semantik pada node AST. Pendekatan ini membuat semantic analyzer menjadi lebih modular, terstruktur, dan mudah dikembangkan.

BAB II

Perancangan & Implementasi

2.1. Perancangan Program

2.1.1. Teknologi yang Digunakan

- Program diimplementasikan menggunakan C++17 dengan compiler g++
- Build system menggunakan Makefile untuk kompilasi seluruh file .cpp di folder src/

2.1.2. Struktur Program

```
src/
├── main.cpp
├── ArionLexer.hpp / .cpp
├── KeywordIdentifierLexer.hpp / .cpp
├── TipeDataKomentar.hpp / .cpp
├── OperatorDelimiterLexer.hpp / .cpp
├── Parser.hpp / .cpp
├── SemanticAnalyzer.hpp / .cpp
├── ASTDecoratedPrinter.hpp / .cpp
├── ASTNode.hpp
└── SymbolTable.hpp
```

2.1.3. Alur Kerja Program

Tahap semantic analysis dimulai setelah parser berhasil menghasilkan parse tree. Berikut adalah alur kerja programnya.

1. Inisialisasi symbol table
Objek SemanticAnalyzer dibuat. Di dalam konstruktor, fungsi initPredefined() dipanggil untuk mengisi tabel simbol awal (tab, btab, atab) dengan reserved words (32 kata kunci, termasuk tipe dasar integer, real, boolean, char, string) dan redefined identifier (true, false, writeln, readln). Tabel tab diindeks mulai dari 33 (0–32 untuk reserved word). Tabel btab diisi dengan blok global (indeks 0), dan display menyimpan tingkat scope.
2. Traversal parse tree dengan visitor functions
Fungsi analyze(root, outFilename) dipanggil dengan root parse tree. Dilakukan penelusuran DFS menggunakan fungsi visit_<node> yang

sesuai dengan jenis node (misal visitProgram, visitDeclarationPart, visitExpression, dll.).

3. Konversi parse tree menjadi decorated AST

Meskipun secara struktural parse tree tetap dipertahankan (label seperti <program> masih ada), node-node tersebut telah dilengkapi dengan informasi semantik.

4. Output semantic analysis

Setelah traversal selesai, program mencetak Decorated AST (dengan anotasi [type:..., tab:..., lev:...]) ke terminal dan ke file ast_output_n.txt. Selanjutnya dicetak isi symbol table (tab, btab, atab) dalam format tabel. Jika ditemukan error semantik (misal tipe tidak kompatibel, identifiier tidak dideklarasikan), program menampilkan pesan error dan menandai hasError = true, tetapi tetap melanjutkan analisis (tidak crash). Di akhir, ditampilkan status sukses atau gagal.

2.1.4. Justifikasi Pilihan Implementasi

Keputusan	Pilihan	Alasan
Pendekatan traversal AST	Visitor functions per jenis node (visitProgram, visitExpression, dll.)	Memisahkan algoritma semantik dari struktur AST; mudah menambahkan aturan baru tanpa mengubah kelas node; memungkinkan kode modular dan terorganisir.
Representasi AST	Struct ASTNode dengan atribut semType, tabIdx, lexLevel; mempertahankan label parse tree	Memudahkan pencetakan decorated AST dan pelacakan anotasi; atribut langsung melekat pada node sehingga tidak perlu tabel anotasi terpisah.
Struktur symbol table	Tiga tabel terpisah (tab, btab, atab) dengan field link untuk rantai per scope	Mengikuti spesifikasi yang mewajibkan tiga tabel; memisahkan informasi identifiier, blok, dan array sehingga mudah dikelola dan diperluas untuk tipe

		komposit.
Manajemen scope	Vector display menyimpan indeks btab per level + field lev di tab; scope bertumpuk via enterBlock() / leaveBlock()	Menjamin pencarian identifier sesuai lexical scoping (dari lokal ke global); operasi push/pop O(1) dan mudah diimplementasikan.
Inisialisasi predefined identifier	Reserved words (0-32) diisi di awal; true, false, writeln, readln ditambahkan setelahnya	Memastikan tipe dasar (integer, real, boolean, char, string) dan konstanta boolean tersedia sejak awal; indeks tetap memudahkan debugging.
Type compatibility	Fungsi typesCompatible dan assignCompatible dengan aturan: Integer $\leftrightarrow$ Real untuk operasi; Real := Integer untuk assignment; lainnya harus sama tepat	Sederhana namun mencakup aturan dasar spesifikasi (subrange belum diterapkan penuh); mudah dikembangkan jika nanti perlu aturan lebih kompleks.
Error handling semantik	semanticError mencatat error dan melanjutkan (tidak throw); semanticWarning untuk peringatan	Memungkinkan pengumpulan semua error dalam satu kali run (bukan stop di error pertama); user mendapat laporan lengkap; program tetap berjalan untuk memberi anotasi maksimal.
Dekorasi AST (anotasi)	Node langsung diberi atribut semType, tabIdx, lexLevel saat dikunjungi	Menghasilkan decorated AST tanpa perlu struktur paralel; memudahkan pencetakan anotasi bersama pohon.
Penanganan tipe enumerasi	Setiap identifier dalam (a,b,c) didaftarkan sebagai OBJ_CONST dengan nilai posisi	Sederhana dan sesuai kebutuhan pengecekan tipe; nilai ordinal dapat digunakan nanti untuk

	(0,1,2,...)	code generation.
Pengecekan tipe subrange	Batas subrange diekstrak dan diperiksa ($\text{low} \leq \text{high}$) serta tipe kedua batas harus sama; dilarang Real	Memenuhi aturan dasar spesifikasi; mencegah range tidak valid sejak awal analisis.
Output decorated AST dan symbol table	Kelas terpisah ASTDecoratedPrinter	Memisahkan tanggung jawab pencetakan dari logika analisis semantik, meningkatkan modularitas dan memudahkan pemeliharaan.

2.2. Implementasi

2.2.1. ASTNode.hpp

```
#ifndef ASTNODE_HPP
#define ASTNODE_HPP

#include <string>
#include <vector>

enum TypeCode {
    TYPE_VOID = 0,
    TYPE_INTEGER = 1,
    TYPE_REAL = 2,
    TYPE_BOOLEAN = 3,
    TYPE_CHAR = 4,
    TYPE_STRING = 5,
    TYPE_ARRAY = 6,
    TYPE_RECORD = 7,
    TYPE_SUBRANGE = 8,
    TYPE_ENUMERATED = 9,
    TYPE_UNKNOWN = -1
};

struct ASTNode {
    std::string label;
    bool isTerminal;
    std::string tokenType;
    std::string tokenValue;

    std::vector<ASTNode*> children;

    int semType;
    int tabIdx;
    int lexLevel;

    ASTNode(const std::string& lbl, bool term = false, const std::string& tt = "", const std::string& tv = "")
        : label(lbl), isTerminal(term), tokenType(tt), tokenValue(tv), semType(TYPE_VOID), tabIdx(-1), lexLevel(0) {}

    ~ASTNode() {
        for (auto* c : children) delete c;
    }

    void addChild(ASTNode* c) {
        if (c) children.push_back(c);
    }
};

#endif
```

Nama	Tipe	Deskripsi
label	std::string	Label node, misalnya "<program>" atau "ident".
isTerminal	bool	true jika node adalah token terminal (misal identifier, angka,

		operator).
tokenType	std::string	Tipe token (hanya untuk terminal), contoh: "ident", "intcon".
tokenValue	std::string	Nilai token (hanya untuk terminal), contoh: "Hello", "100".
children	std::vector<ASTNode*>	Daftar child node.
semType	int	Kode tipe data hasil evaluasi semantik (dari enum TypeCode).
tabIdx	int	Indeks entri di tabel simbol tab (atau -1 jika tidak ada).
lexLevel	int	Tingkat lexical scope (0 = global, 1 = di dalam prosedur, dst.).
ASTNode(...)	Constructor	Membuat node baru dengan label, flag terminal, tipe token, nilai token.
addChild(ASTNode*)	void	Menambahkan child ke node.

2.2.2. SymbolTable.hpp

```
#ifndef SYMBOLTABLE_HPP
#define SYMBOLTABLE_HPP

#include <string>

enum ObjKind {
    OBJ_CONST    = 0,
    OBJ_VAR      = 1,
    OBJ_TYPE     = 2,
    OBJ_PROC     = 3,
    OBJ_FUNC     = 4,
    OBJ_PROGRAM  = 5,
    OBJ_RESERVED = 6
};

struct TabEntry {
    std::string id;
    int link;
    int obj;
    int type;
    int ref;
    int nrm;
    int lev;
    int adr;
};

struct BtabEntry {
    int last;
    int lpar;
    int psze;
    int vsze;
};

struct AtabEntry {
    int xtyp;
    int etyp;
    int eref;
    int low;
    int high;
    int elsz;
    int size;
};

#endif
```

Struct TabEntry

Field	Tipe	Deskripsi
id	std::string	Nama identifier (case-sensitive disimpan asli, perbandingan case-insensitive).
link	int	Indeks ke entri sebelumnya dalam scope yang sama (linked list per blok).
obj	int	Jenis objek (OBJ_CONST, OBJ_VAR, OBJ_TYPE, OBJ_PROC, OBJ_FUNC, OBJ_PROGRAM, OBJ_RESERVED).
type	int	Kode tipe data (dari enum TypeCode).
ref	int	Pointer ke tabel lain: indeks di atab (untuk array) atau btab (untuk record/prosedur).
nrm	int	1 untuk variabel normal, 0 untuk parameter by-reference.
lev	int	Tingkat lexical scope tempat identifier dideklarasikan.
adr	int	Alamat (offset) untuk variabel, atau nilai untuk konstanta.

Struct BtabEntry

Field	Tipe	Deskripsi
last	int	Indeks di tab dari identifier terakhir yang dideklarasikan dalam blok ini.

lpar	int	Indeks parameter terakhir (0 jika bukan prosedur/fungsi).
psze	int	Total ukuran parameter dalam unit memori.
vsze	int	Total ukuran variabel lokal dalam unit memori.

Struct AtabEntry

Field	Tipe	Deskripsi
xtyp	int	Kode tipe indeks array (misal TYPE_INTEGER).
etyp	int	Kode tipe elemen array.
eref	int	Pointer ke tipe elemen jika elemen adalah tipe komposit (array/record).
low	int	Batas bawah indeks array.
high	int	Batas atas indeks array.
elsz	int	Ukuran satu elemen (dalam unit).
size	int	Total ukuran array = $(\text{high} - \text{low} + 1) \times \text{elsz}$.

2.2.3. SemanticAnalyzer.hpp/.cpp

```
#ifndef SEMANTICANALYZER_HPP
#define SEMANTICANALYZER_HPP

#include "ASTNode.hpp"
#include "SymbolTable.hpp"
#include <vector>
#include <string>
#include <fstream>

class SemanticAnalyzer {
public:
    SemanticAnalyzer();
    ~SemanticAnalyzer();

    void analyze(ASTNode* root, const std::string& outFilename);

private:
    // Symbol tables
    std::vector<TabEntry> tab;
    std::vector<BtabEntry> btab;
    std::vector<AtabEntry> atab;

    std::vector<int> display;
    int currentLevel;
    int mainBlockIndex;

    std::ofstream outFile;
    bool hasError;

    // Initialisation
    void initPredefined();

    // Scope / symbol management
    int ensureMainBlock();
    int enterBlock();
    void leaveBlock();
    int addIdentifier(const std::string& name, int obj, int type, int ref, int nrm, int adr);
    int lookupIdent(const std::string& name, bool errorIfMissing = true);
    int resolveTypeName(const std::string& name);
    int getTypeSize(int typeCode, int ref) const;
```

```

// Visitor functions
void visitProgram(ASTNode* node);
void visitDeclarationPart(ASTNode* node);
void visitConstDeclaration(ASTNode* node);
int visitConstant(ASTNode* node);
void visitTypeDeclaration(ASTNode* node);
int visitType(ASTNode* node, int& outRef);
int visitArrayType(ASTNode* node, int& outRef);
int visitRange(ASTNode* node, int& outRef, int* low = nullptr, int* high = nullptr);
int visitEnumerated(ASTNode* node, int& outRef);
int visitRecordType(ASTNode* node, int& outRef);
void visitFieldList(ASTNode* node);
void visitFieldPart(ASTNode* node);
void visitVarDeclaration(ASTNode* node);
void visitSubprogramDeclaration(ASTNode* node);
void visitProcedureDeclaration(ASTNode* node);
void visitFunctionDeclaration(ASTNode* node);
void visitFormalParameterList(ASTNode* node);
void visitParameterGroup(ASTNode* node);
void visitBlock(ASTNode* node);
void visitCompoundStatement(ASTNode* node);
void visitStatementList(ASTNode* node);
void visitStatement(ASTNode* node);
void visitAssignmentStatement(ASTNode* node);
void visitIfStatement(ASTNode* node);
void visitCaseStatement(ASTNode* node);
void visitCaseBlock(ASTNode* node);
void visitWhileStatement(ASTNode* node);
void visitRepeatStatement(ASTNode* node);
void visitForStatement(ASTNode* node);
void visitProcFuncCall(ASTNode* node);
int visitExpression(ASTNode* node);
int visitSimpleExpression(ASTNode* node);
int visitTerm(ASTNode* node);
int visitFactor(ASTNode* node);
int visitVariable(ASTNode* node);
int visitComponentVariable(ASTNode* node, int parentType, int parentRef, int& newRef);
void visitIndexList(ASTNode* node);

```

```

// Type helpers
bool typesCompatible(int t1, int t2) const;
bool assignCompatible(int targetType, int valueType) const;
bool isRelOp(const std::string& tokenType) const;
bool isNumeric(int typeCode) const;

void emit(const std::string& s);

std::string typeCodeName(int t) const;
std::string objKindName(int o) const;
std::string toLower(const std::string& s) const;

// Error
void semanticError(const std::string& msg);
void semanticWarning(const std::string& msg);
};

#endif

```

Atribut utama

Nama	Tipe	Deskripsi
tab	std::vector<TabEntry>	Tabel identifier utama.
btabs	std::vector<BtabEntry>	Tabel blok (untuk prosedur, fungsi, record).
atab	std::vector<AtabEntry>	Tabel array.
display	std::vector<int>	Menyimpan indeks btabs untuk setiap tingkat lexical level.
currentLevel	int	Tingkat lexical level saat ini (0 = global).
mainBlockIndex	int	Indeks blok utama program di btabs (untuk compound-statement utama). Diinisialisasi -1.
outFile	std::ofstream	File output untuk menyimpan hasil analisis.
hasError	bool	Menandakan apakah ada

		error semantik selama analisis.
--	--	---------------------------------

Metode utama (publik)

Nama	Return	Deskripsi
SemanticAnalyzer()	-	Konstruktor; memanggil initPredefined().
analyze(ASTNode*, const std::string&)	void	Titik masuk utama: melakukan traversal, dekorasi, dan output.

Metode private (pengelolaan simbol)

Nama	Return	Deskripsi
initPredefined()	void	Mengisi tabel dengan reserved words dan predefined identifier (true, false, writeln, readln).
ensureMainBlock()	int	Memastikan blok utama program (level 1) telah dibuat di btab; membuatnya jika belum, lalu mengembalikan indeks blok.
enterBlock()	int	Memasuki blok baru (prosedur/fungsi/record): menambah btab, menaikkan currentLevel, mengembalikan indeks blok.
leaveBlock()	void	Keluar dari blok akan menurunkan currentLevel.
addIdentifier(...)	int	Menambahkan identifier ke tab; memeriksa duplikasi dalam scope yang sama.
lookupIdent(name,	int	Mencari identifier mulai

errorIfMissing)		dari scope terdalam; mengembalikan indeks atau -1.
resolveTypeName(name)	int	Mencari tipe berdasarkan nama; memastikan objek adalah OBJ_TYPE.
getTypeSize(typeCode, ref)	int	Menghitung ukuran memori dari suatu tipe (1 untuk skalar, ukuran array/record sesuai).

Metode visitor

Metode Visitor	Parameter	Deskripsi
visitProgram	ASTNode* node	Mengunjungi node root <program>. Mengambil nama program dari header, menambahkannya ke tab sebagai OBJ_PROGRAM, lalu memanggil visitor untuk bagian deklarasi dan compound statement.
visitDeclarationPart	ASTNode* node	Mengunjungi <declaration-part>. Memanggil visitor untuk setiap child: visitConstDeclaration, visitTypeDeclaration, visitVarDeclaration, visitSubprogramDeclarat ion sesuai jenisnya.
visitConstDeclaration	ASTNode* node	Mengunjungi <const-declaration>. Mengiterasi pasangan ident == constant,, memanggil visitConstant untuk mendapatkan tipe, lalu menambahkan konstanta ke tab sebagai

		OBJ_CONST.
visitConstant	ASTNode* node	Mengunjungi <constant>. Menentukan tipe berdasarkan child terminal (charcon, string, intcon, realcon, atau ident yang sudah dideklarasikan). Mengembalikan kode tipe.
visitTypeDeclaration	ASTNode* node	Mengunjungi <type-declaration>. Mengiterasi ident == type;, memanggil visitType untuk mendapatkan tipe dan ref, lalu menambahkan tipe ke tab sebagai OBJ_TYPE.
visitType	ASTNode* node, int& outRef	Mengunjungi <type>. Menentukan apakah tipe adalah ident sederhana, array, range, enumerated, atau record. Memanggil visitor spesifik dan mengisi outRef. Mengembalikan kode tipe.
visitArrayType	ASTNode* node, int& outRef	Mengunjungi <array-type>. Membaca indeks (dari range atau ident) dan tipe elemen. Membuat entri di atab, mengisi xtyp, etyp, low, high, elsz, size. outRef diisi indeks atab.
visitRange	ASTNode* node, int& outRef, int* low=nullptr, int* high=nullptr	Mengunjungi <range>. Memeriksa tipe batas (tidak boleh Real) dan bahwa $low \leq high$. Ekstrak nilai integer jika diperlukan. outRef tidak

		terpakai (0).
visitEnumerated	ASTNode* node, int& outRef	Mengunjungi <enumerated>. Setiap ident didaftarkan sebagai OBJ_CONST dengan tipe TYPE_ENUMERATED dan nilai posisi (0,1,2,...). outRef = 0.
visitRecordType	ASTNode* node, int& outRef	Mengunjungi <record-type>. Memanggil enterBlock() untuk membuat blok record, lalu visitFieldList, kemudian leaveBlock(). outRef diisi indeks btab dari blok record.
visitFieldList	ASTNode* node	Mengunjungi <field-list>. Memanggil visitFieldPart untuk setiap child <field-part>.
visitFieldPart	ASTNode* node	Mengunjungi <field-part>. Mengambil <identifier-list> dan <type>, menghitung tipe dan ukuran, lalu menambahkan setiap identifier sebagai OBJ_VAR dengan offset sesuai btab[display[currentLevel]].vsze.
visitVarDeclaration	ASTNode* node	Mengunjungi <var-declaration>. Mengiterasi kelompok identifier-list : type ;, memanggil visitType, menghitung offset dan ukuran, menambahkan variabel ke tab sebagai OBJ_VAR, dan

		menganotasi node ident.
visitSubprogramDeclaration	ASTNode* node	Mengunjungi <subprogram-declaration>. Memanggil visitProcedureDeclaration atau visitFunctionDeclaration sesuai jenis.
visitProcedureDeclaration	ASTNode* node	Mengunjungi <procedure-declaration>. Mencatat nama prosedur, membuat blok baru dengan enterBlock(), memproses parameter (jika ada), memproses blok isi, lalu leaveBlock().
visitFunctionDeclaration	ASTNode* node	Mengunjungi <function-declaration>. Mirip prosedur, tetapi juga mencatat tipe kembalian (setelah colon).
visitFormalParameterList	ASTNode* node	Mengunjungi <formal-parameter-list>. Memanggil visitParameterGroup untuk setiap kelompok parameter.
visitParameterGroup	ASTNode* node	Mengunjungi <parameter-group>. Mengambil <identifier-list> dan tipe (bisa ident atau array-type). Menambahkan parameter ke tab sebagai OBJ_VAR, menghitung offset di btap.psize.
visitBlock	ASTNode* node	Mengunjungi <block>. Memanggil

		visitDeclarationPart dan visitCompoundStatement untuk childnya.
visitCompoundStatement	ASTNode* node	Mengunjungi <compound-statement>. Memanggil visitStatementList jika ada.
visitStatementList	ASTNode* node	Mengunjungi <statement-list>. Memanggil visitStatement untuk setiap child statement.
visitStatement	ASTNode* node	Mengunjungi <statement>. Mendelegasikan ke visitor yang sesuai berdasarkan label node (<assignment-statement>, <if-statement>, dll.) atau visitCompoundStatement jika perlu.
visitAssignmentStatement	ASTNode* node	Mengunjungi assignment. Menentukan tipe target (dari ident atau <variable>) dan tipe ekspresi, lalu memeriksa assignCompatible. Menganotasi node target dan ekspresi.
visitIfStatement	ASTNode* node	Mengunjungi if-statement. Memeriksa bahwa ekspresi kondisi bertipe Boolean, lalu memanggil visitor untuk statement then dan else (jika ada).
visitCaseStatement	ASTNode* node	Mengunjungi case-statement. Memanggil

		visitExpression untuk ekspresi pengendali, lalu visitCaseBlock.
visitCaseBlock	ASTNode* node	Mengunjungi <case-block>. Memanggil visitConstant untuk setiap konstanta label, lalu visitStatement untuk aksi.
visitWhileStatement	ASTNode* node	Mengunjungi while-loop. Memeriksa bahwa kondisi bertipe Boolean, lalu membuat blok scope baru (enterBlock()), memanggil visitor untuk badan loop (compound statement), kemudian keluar dari blok (leaveBlock()).
visitRepeatStatement	ASTNode* node	Mengunjungi repeat-until. Memanggil visitStatementList untuk badan, lalu memeriksa bahwa kondisi bertipe Boolean.
visitForStatement	ASTNode* node	Mengunjungi for-loop. Memastikan variabel pengendali bertipe Integer atau Char dan sudah dideklarasikan, lalu mengevaluasi ekspresi awal dan akhir, membuat blok scope baru (enterBlock()), memanggil visitor untuk badan loop (compound statement), kemudian keluar dari blok (leaveBlock()).
visitProcFuncCall	ASTNode* node	Mengunjungi

		pemanggilan prosedur/fungsi. Mencari nama di tab, memeriksa bahwa objek adalah OBJ_PROC atau OBJ_FUNC, kemudian (jika ada) memanggil visitExpression untuk setiap argumen.
visitExpression	ASTNode* node	Mengunjungi <expression>. Menggabungkan tipe dari simple-expression kiri dan kanan (jika ada operator relasional). Hasil tipe ditentukan dan dikembalikan.
visitSimpleExpression	ASTNode* node	Mengunjungi <simple-expression>. Menangani tanda plus/minus unary, lalu menggabungkan tipe dari term-term yang dihubungkan oleh +, -, atau or. Hasil tipe dikembalikan.
visitTerm	ASTNode* node	Mengunjungi <term>. Menggabungkan tipe dari factor-factor yang dihubungkan oleh *, /, div, mod, atau and. Hasil tipe dikembalikan.
visitFactor	ASTNode* node	Mengunjungi <factor>. Menentukan tipe berdasarkan child: konstanta numerik, string, char, ident, ekspresi dalam kurung, not, atau pemanggilan fungsi. Jika child adalah ident, dilakukan lookup.
visitVariable	ASTNode* node	Mengunjungi

		<variable>. Mencari identifikasi awal di tab, lalu memproses komponen (array index atau field record) dengan visitComponentVariable. Mengembalikan tipe akhir.
visitComponentVariable	ASTNode* node, int parentType, int parentRef, int& newRef	Mengunjungi <component-variable>. Jika node adalah lbrack, memproses indeks (array) dan mengembalikan tipe elemen (dari atab). Jika node adalah period, mencari field dalam record (dari btab). newRef diisi pointer ke tipe komponen.
visitIndexList	ASTNode* node	Mengunjungi <index-list>. Memeriksa setiap ekspresi indeks (bisa konstanta atau ident) dan mencatat referensinya di tab jika perlu.

Metode helper

Nama	Return	Deskripsi
typesCompatible(t1, t2)	bool	Mengecek kompatibilitas tipe untuk operasi biner (Integer & Real kompatibel).
assignCompatible(target, value)	bool	Mengecek kompatibilitas untuk assignment (Real := Integer diperbolehkan).
isRelOp(const std::string&)	bool	Mengecek apakah token adalah operator relasional (eq, neq, lss,

		leq, gtr, geq).
isNumeric(int)	bool	Mengecek apakah tipe adalah numerik (INTEGER atau REAL).
emit(const std::string&)	void	Mencetak pesan ke terminal dan file output (sekarang hanya untuk status, bukan untuk AST/symbol table).
typeCodeName(int)	std::string	Mengembalikan nama tipe (misal "integer") dari kode tipe.
objKindName(int)	std::string	Mengembalikan nama jenis objek ("const", "var", dll.) dari kode objek.
toLower(const std::string&)	std::string	Mengubah string menjadi huruf kecil (case-insensitive lookup).
semanticError(const std::string&)	void	Mencatat error, menandai hasError, tetapi tidak menghentikan eksekusi.
semanticWarning(const std::string&)	void	Mencatat peringatan (tidak mempengaruhi hasError).

2.2.4. ASTDecoratedPrinter.hpp/.cpp

```
#ifndef ASTDECORATEDPRINTER_HPP
#define ASTDECORATEDPRINTER_HPP

#include "ASTNode.hpp"
#include "SymbolTable.hpp"
#include <vector>
#include <string>
#include <fstream>

class ASTDecoratedPrinter {
public:
    ASTDecoratedPrinter(
        const std::vector<TabEntry>& tab,
        const std::vector<BtabEntry>& btab,
        const std::vector<AtabEntry>& atab,
        std::ofstream& outFile
    );

    // Main entry
    void printAll(ASTNode* root);

private:
    const std::vector<TabEntry>& _tab;
    const std::vector<BtabEntry>& _btab;
    const std::vector<AtabEntry>& _atab;
    std::ofstream& _out;

    // Emit
    void emit(const std::string& s);
    std::string ind(int d) const; // d*2 spaces

    // Type /object name helpers
    std::string typeFull(int t) const;
    std::string typeShort(int t) const;
    std::string objClass(int o) const;
    std::string opSym(const std::string& tok) const;
    std::string inlineExpr(ASTNode* n) const;
    bool isRelOp(const std::string& tok) const;
    bool isAddOp(const std::string& tok) const;
    bool isMulOp(const std::string& tok) const;
```

```

// Parse-tree navigation helpers
ASTNode* childNT(ASTNode* n, const std::string& label) const;
ASTNode* childTerm(ASTNode* n, const std::string& tok) const;
std::string progName(ASTNode* programNode) const;
std::string stripQuotes(const std::string& s) const;

```

```

// Decorated AST printers
void doProg(ASTNode* n, int d);
void doDecls(ASTNode* n, int d);
void doVarDecl(ASTNode* n, int d, bool isParam = false);
void doConstDecl(ASTNode* n, int d);
void doTypeDecl(ASTNode* n, int d);
void doRecordFields(int blockIdx, int d);
void doSubprog(ASTNode* n, int d);
void doProcDecl(ASTNode* n, int d);
void doFuncDecl(ASTNode* n, int d);
void doBlock(ASTNode* n, int d);
void doCompound(ASTNode* n, int d);
void doStmtList(ASTNode* n, int d);
void doStmt(ASTNode* n, int d);
void doAssign(ASTNode* n, int d);
void doIf(ASTNode* n, int d);
void doWhile(ASTNode* n, int d);
void doFor(ASTNode* n, int d);
void doRepeat(ASTNode* n, int d);
void doCase(ASTNode* n, int d);
void doCaseBlock(ASTNode* n, int d);
void doProcCall(ASTNode* n, int d);

```

```

void doExpr(ASTNode* n, int d);
void doFactor(ASTNode* n, int d);
void doVarRef(ASTNode* n, int d);
void doConstantNode(ASTNode* n, int d);
void doBinChain(
    const std::vector<ASTNode*>& operands,
    const std::vector<std::string>& opToks,
    int resultType,
    int d);

```

```

// Symbol-table printers
void doTab();
void doBtab();
void doAtab();
};

#endif

```

Atribut

Nama	Tipe	Deskripsi
_tab	const std::vector<TabEntry>&	Referensi ke tabel identifier (dari SemanticAnalyzer).
_btabs	const std::vector<BtabEntry>&	Referensi ke tabel blok (prosedur/fungsi/record).
_atabs	const std::vector<AtabEntry>&	Referensi ke tabel array.
_out	std::ofstream&	Referensi ke file output (bisa dibuka/ditutup di luar).

Metode public

Nama	Return	Deskripsi
ASTDecoratedPrinter(tab, btabs, atabs, outFile)	-	Konstruktor, menyimpan referensi ke tabel dan file output.
printAll(root)	void	Titik masuk utama yang mencetak decorated AST (dimulai dari doProg), lalu memanggil doTab(), doBtabs(), dan doAtabs() jika ada.

Metode privat

Nama	Return	Deskripsi
emit(s)	void	Mencetak string s ke terminal dan file output.
ind(d)	std::string	Mengembalikan spasi sebanyak d*2 (untuk indentasi).
typeFull(t)	std::string	Nama tipe lengkap ("integer", "real", dll.).

typeShort(t)	std::string	Nama tipe pendek ("INT", "REAL", dll.) untuk tabel.
objClass(o)	std::string	Nama kelas objek ("CONST", "VAR", dll.) dari ObjKind.
opSym(tok)	std::string	Mengubah token operator ke simbol (misal "plus" $\rightarrow$ "+").
inlineExpr(n)	std::string	Menghasilkan representasi satu baris untuk ekspresi (untuk assignment).
isRelOp(tok), isAddOp(tok), isMulOp(tok)	bool	Mengecek apakah token termasuk operator relasional, aditif, atau multiplikatif.
childNT(n, label)	ASTNode*	Mencari child non-terminal dengan label tertentu.
childTerm(n, tok)	ASTNode*	Mencari child terminal dengan tipe token tertentu.
progName(programNode)	std::string	Mengambil nama program dari node <program>.
stripQuotes(s)	std::string	Menghapus tanda petik tunggal di awal/akhir string.
doProg(n, d)	void	Mencetak ProgramNode beserta deklarasi dan compound statement.
doDecls(n, d)	void	Mencetak semua deklarasi (konstanta, tipe, variabel, subprogram).
doVarDecl(n, d,	void	Mencetak deklarasi

isParam)		variabel (atau parameter jika isParam=true).
doConstDecl(n, d)	void	Mencetak deklarasi konstanta dan nilainya.
doTypeDecl(n, d)	void	Mencetak deklarasi tipe; jika record, juga field-fieldnya.
doRecordFields(blockId x, d)	void	Mencetak field-field dari record berdasarkan indeks blok.
doSubprog(n, d)	void	Mencetak subprogram (prosedur/fungsi).
doProcDecl(n, d)	void	Mencetak prosedur, parameter (jika ada), dan blok isinya.
doFuncDecl(n, d)	void	Mencetak fungsi, tipe kembalian, parameter, dan blok.
doBlock(n, d)	void	Mencetak blok (deklarasi lokal + compound statement).
doCompound(n, d)	void	Mencetak compound statement (blok kode).
doStmtList(n, d)	void	Mencetak daftar statement.
doStmt(n, d)	void	Mencetak statement sesuai jenisnya (delegasi ke doAssign, doIf, dll.).
doAssign(n, d)	void	Mencetak assignment: Assign('x' := ...), target, nilai.
doIf(n, d)	void	Mencetak If, kondisi, cabang then/else.
doWhile(n, d)	void	Mencetak While, kondisi, body.

doFor(n, d)	void	Mencetak For, variabel, arah (to/downto), start, end, body.
doRepeat(n, d)	void	Mencetak Repeat, body, kondisi until.
doCase(n, d)	void	Mencetak Case, ekspresi, dan cabang-cabang case-block.
doCaseBlock(n, d)	void	Mencetak satu cabang case: label(s) diikuti statement.
doProcCall(n, d)	void	Mencetak pemanggilan prosedur/fungsi dan argumennya.
doExpr(n, d)	void	Mencetak ekspresi (mendeteksi operator biner/unary).
doFactor(n, d)	void	Mencetak faktor (konstanta, variabel, not, kurung, dll.).
doVarRef(n, d)	void	Mencetak referensi variabel (termasuk komponen array/record).
doConstantNode(n, d)	void	Mencetak konstanta (angka, char, string, ident).
doBinChain(operands, opToks, resultType, d)	void	Mencetak rantai operator biner asosiatif kiri (left-fold).
doTab()	void	Mencetak tabel tab dalam format tabel rapi.
doBtab()	void	Mencetak tabel btab.
doAtab()	void	Mencetak tabel atab (jika tidak kosong).

2.3. Semantic Action

Production Rule	Semantic Action
$\langle \text{program} \rangle \rightarrow \langle \text{program-header} \rangle$ $\langle \text{declaration-part} \rangle$ $\langle \text{compound-statement} \rangle \text{ period}$	visitProgram: mencatat nama program ke tab (OBJ_PROGRAM), lalu mengunjungi deklarasi dan compound statement. Anotasi node: semType = TYPE_VOID, tabIdx = indeks program.
$\langle \text{program-header} \rangle \rightarrow \text{programsy ident semicolon}$	visitProgramHeader (dipanggil dari visitProgram): ekstrak nama program.
$\langle \text{declaration-part} \rangle \rightarrow \{$ $\langle \text{const-declaration} \rangle \mid \langle \text{type-declaration} \rangle \mid$ $\langle \text{var-declaration} \rangle \mid$ $\langle \text{subprogram-declaration} \rangle \}^*$	visitDeclarationPart: memanggil visitor sesuai jenis child.
$\langle \text{const-declaration} \rangle \rightarrow \text{constsy (ident eql } \langle \text{constant} \rangle \text{ semicolon)}^+$	visitConstDeclaration: untuk setiap ident, memanggil visitConstant untuk mendapatkan tipe, lalu addIdentifier(ident, OBJ_CONST, type, 0, 1, 0).
$\langle \text{constant} \rangle \rightarrow \text{charcon} \mid \text{string} \mid$ $[\text{plus} \mid \text{minus}]? (\text{ident} \mid \text{intcon} \mid \text{realcon})$	visitConstant: menentukan tipe dari child terminal (charcon $\rightarrow$ TYPE_CHAR, string $\rightarrow$ TYPE_STRING, intcon $\rightarrow$ TYPE_INTEGER, realcon $\rightarrow$ TYPE_REAL, ident $\rightarrow$ lookup tipe). Anotasi node dan child semType.
$\langle \text{type-declaration} \rangle \rightarrow \text{typesy (ident eql } \langle \text{type} \rangle \text{ semicolon)}^+$	visitTypeDeclaration: untuk setiap ident, memanggil visitType (mengisi outRef), lalu addIdentifier(ident, OBJ_TYPE, typeCode, ref, 1, 0).
$\langle \text{type} \rangle \rightarrow \langle \text{simple-type} \rangle \mid \langle \text{array-type} \rangle \mid$ $\langle \text{record-type} \rangle$	visitType: delegasi ke visitor sesuai child, isi outRef.
$\langle \text{array-type} \rangle \rightarrow \text{arraysy lbrack (} \langle \text{range} \rangle \mid$ $\text{ident) rbrack ofsy } \langle \text{type} \rangle$	visitArrayType: buat entri AtabEntry, isi xtyp (dari range atau ident), etyp (dari tipe elemen), low, high, elsz, size. outRef = indeks atab.
$\langle \text{range} \rangle \rightarrow \langle \text{constant} \rangle \text{ period period } \langle \text{constant} \rangle$	visitRange: periksa tipe batas (tidak boleh TYPE_REAL) dan $\text{low} \leq \text{high}$. Ekstrak nilai integer jika ada. Anotasi semType = tipe batas.

<enumerated> → lparent ident (comma ident)* rparent	visitEnumerated: setiap ident ditambahkan sebagai OBJ_CONST dengan tipe TYPE_ENUMERATED dan adr = posisi (0,1,2,...).
<record-type> → recordsy <field-list> endsy	visitRecordType: memanggil enterBlock(), visitFieldList(), leaveBlock(). outRef = indeks blok btab.
<field-list> → <field-part> (semicolon <field-part>)*	visitFieldList: memanggil visitFieldPart untuk setiap child <field-part>.
<field-part> → <identifier-list> colon <type>	visitFieldPart: memanggil visitType, hitung ukuran tipe, tambahkan setiap ident di identifier-list sebagai OBJ_VAR dengan offset btab[display[currentLevel]].vsze (diperbarui).
<var-declaration> → varsy (<identifier-list> colon <type> semicolon)+	visitVarDeclaration: sama seperti field part, tetapi untuk variabel global/lokal (bukan field record).
<subprogram-declaration> → <procedure-declaration> <function-declaration>	visitSubprogramDeclaration: delegasi.
<procedure-declaration> → proceduresy ident [<formal-parameter-list>] semicolon <block> semicolon	visitProcedureDeclaration: tambahkan ident ke tab sebagai OBJ_PROC, memanggil enterBlock(), proses parameter, proses blok, leaveBlock().
<function-declaration> → functionsy ident [<formal-parameter-list>] colon ident semicolon <block> semicolon	visitFunctionDeclaration: sama seperti prosedur, tetapi tipe kembalian diambil dari ident setelah colon.
<formal-parameter-list> → lparent <parameter-group> (semicolon <parameter-group>)* rparent	visitFormalParameterList: memanggil visitParameterGroup untuk setiap grup.
<parameter-group> → <identifier-list> colon (<array-type> ident)	visitParameterGroup: tambahkan setiap ident sebagai OBJ_VAR dengan offset btab[display[currentLevel]].psze (diperbarui).
<compound-statement> → beginsy <statement-list> endsy	visitCompoundStatement: memanggil visitStatementList.

<statement-list> → (<statement> (semicolon <statement>)*)?	visitStatementList: memanggil visitStatement untuk setiap statement.
<assignment-statement> → (<variable> ident) becomes <expression>	visitAssignmentStatement: menentukan tipe target (dari ident atau variable) dan tipe ekspresi, lalu periksa assignCompatible. Anotasi target dan ekspresi.
<if-statement> → ifsy <expression> then sy <statement> [else sy <statement>]	visitIfStatement: periksa bahwa tipe ekspresi adalah TYPE_BOOLEAN, lalu mengunjungi statement cabang.
<while-statement> → while sy <expression> dosy <compound-statement> semicolon	visitWhileStatement: periksa tipe ekspresi Boolean, mengunjungi body.
<repeat-statement> → repeatsy <statement-list> until sy <expression>	visitRepeatStatement: mengunjungi body, periksa tipe ekspresi Boolean.
<for-statement> → forsy ident becomes <expression> (tosy downtosy) <expression> dosy <compound-statement> semicolon	visitForStatement: periksa ident (harus variabel Integer/Char), mengunjungi kedua ekspresi, mengunjungi body.
<procedure/function-call> → ident [lparent [<parameter-list>] rparent]	visitProcFuncCall: lookup ident, pastikan OBJ_PROC atau OBJ_FUNC, lalu mengunjungi parameter.
<expression> → <simple-expression> [(eq neq gtr geq lss leq) <simple-expression>]	visitExpression: mengunjungi simple-expression kiri, jika ada operator relasional maka mengunjungi kanan, periksa kompatibilitas tipe, hasil TYPE_BOOLEAN.
<simple-expression> → [plus minus] <term> { (plus minus orsy) <term> }*	visitSimpleExpression: proses tanda unary, mengunjungi term pertama, lalu untuk setiap operator: or → periksa Boolean, hasil Boolean; +/- → periksa numerik, hasil Integer/Real.
<term> → <factor> { (times rdiv idiv imod andsy) <factor> }*	visitTerm: mengunjungi factor pertama, lalu untuk setiap operator: and → periksa Boolean, hasil Boolean; div/mod → periksa Integer, hasil Integer; rdiv → numerik, hasil Real; * → numerik, hasil Integer/Real.

<factor> → intcon realcon charcon string lparent <expression> rparent notsy <factor> ident <procedure/function-call> <variable>	visitFactor: menentukan tipe dari child: konstanta langsung (intcon→Integer, dll.), not → periksa Boolean, hasil Boolean; ident → lookup; ekspresi dalam kurung → hasil tipe ekspresi; prosedur/fungsi → tipe sesuai definisi; variable → tipe variabel.
<variable> → ident { <component-variable> }*	visitVariable: lookup ident awal, kemudian untuk setiap komponen (array index atau field) memanggil visitComponentVariable. Hasil tipe akhir.
<component-variable> → lbrack <index-list> rbrack period ident	visitComponentVariable: jika array, ambil tipe elemen dari atab; jika record, cari field di btab.

2.4. Predefined Identifier

Identifier	Jenis (obj)	Type Data	Nilai / Keterangan
integer	tipe	TYPE_INTEGER	Tipe dasar integer
real	tipe	TYPE_REAL	Tipe dasar real
boolean	tipe	TYPE_BOOLEAN	Tipe dasar boolean
char	tipe	TYPE_CHAR	Tipe dasar karakter
string	tipe	TYPE_STRING	Tipe dasar string
true	konstanta	TYPE_BOOLEAN	Nilai 1 (true)
false	konstanta	TYPE_BOOLEAN	Nilai 0 (false)
writeln	prosedur	TYPE_VOID	Prosedur standar output
readln	prosedur	TYPE_VOID	Prosedur standar input

Selain itu, 32 reserved words (keyword) dimasukkan ke dalam tab dengan obj = OBJ_RESERVED untuk keyword murni (seperti and, begin, if, while, dll.) dan obj = OBJ_TYPE untuk tipe dasar (integer, real, boolean, char, string). Keyword murni memiliki type = TYPE_VOID, sedangkan tipe dasar memiliki type yang sesuai (TYPE_INTEGER, TYPE_REAL, TYPE_BOOLEAN, TYPE_CHAR, TYPE_STRING). Reserved words tersebut meliputi: and, array, begin, case, const, div, downto, do, else,

end, for, function, if, mod, not, of, or, procedure, program, record, repeat, then, to, type, until, var, while, serta tipe dasar integer, real, boolean, char, string.

2.5. Type Compatibility Rule

2.5.1. Type Compatibility (untuk operasi biner, relasional, aritmatika)

Dua tipe t_1 dan t_2 dinyatakan compatible jika memenuhi salah satu:

- $t_1 == t_2$ (sama persis)
- t_1 dan t_2 keduanya numerik (TYPE_INTEGER atau TYPE_REAL). Ini memungkinkan campuran Integer dan Real dalam operasi aritmatika dan perbandingan.

2.5.2. Assignment Compatibility (untuk statement assignment)

Suatu nilai bertipe valueType dapat diberikan ke variabel bertipe targetType jika memenuhi salah satu:

- $targetType == valueType$
- $targetType == TYPE_REAL$ dan $valueType == TYPE_INTEGER$ (konversi implisit Integer ke Real)
- $targetType == TYPE_UNKNOWN$ atau $valueType == TYPE_UNKNOWN$ (tidak dilakukan pengecekan, untuk menghindari double error)

2.5.3. Aturan Tambahan (diimplementasikan di berbagai visitor)

- Kondisi percabangan/loop: Ekspresi pada if, while, repeat harus bertipe TYPE_BOOLEAN. Jika tidak, error semantik.
- Operator and / or / not: Operand harus bertipe TYPE_BOOLEAN.
- Operator div / mod: Operand harus bertipe TYPE_INTEGER.
- Operator rdiv (/): Operand harus numerik (Integer atau Real). Hasil selalu TYPE_REAL.
- Operator aritmatika (+, -, *): Operand harus numerik. Hasil TYPE_INTEGER jika kedua operand Integer, sebaliknya TYPE_REAL.

- Indeks array: Tipe indeks tidak boleh TYPE_REAL. Diperbolehkan TYPE_INTEGER, TYPE_CHAR, TYPE_ENUMERATED, atau subrange darinya.
- Subrange (range): Batas bawah dan atas harus memiliki tipe yang sama, dan tidak boleh TYPE_REAL. Batas bawah tidak boleh lebih besar dari batas atas.
- Record field access: Nama field harus terdefinisi dalam blok record yang sesuai.
- Anonymous record: Meskipun strukturnya sama, dua anonymous record dianggap tidak compatible (tidak diimplementasikan secara eksplisit, tetapi karena tipe record yang berbeda memiliki ref berbeda, maka typesCompatible akan mengembalikan false kecuali $t1 == t2$, dengan demikian secara implisit tidak compatible).

BAB III

Pengujian

3.1. Test Case 1

Input
<pre>program Test1; var x, y: integer; r: real; begin x := 10; y := x + 5; r := x; { assignment-compatible: real := integer } writeln(y); end.</pre>
Output
DECORATED AST: ----- ProgramNode(name: 'Test1', tab_index: 37) Declarations: - VarDecl(name: 'x', type: integer, tab_index: 38, level: 0) - VarDecl(name: 'y', type: integer, tab_index: 39, level: 0) - VarDecl(name: 'r', type: real, tab_index: 40, level: 0) Block(block_index: 1, lev: 1) Assign('x' := 10, type: void) Target: - Var(name: 'x', tab_index: 38, type: integer) Value: - Number(int: 10, type: integer) Assign('y' := x+5, type: void) Target: - Var(name: 'y', tab_index: 39, type: integer) Value: - BinOp(op: '+', type: integer) - Left: - Var(name: 'x', tab_index: 38, type: integer)

```

Right:
  Number(int: 5, type: integer)
Assign('r' := x, type: void)
Target:
  Var(name: 'r', tab_index: 40, type: real)
Value:
  Var(name: 'x', tab_index: 38, type: integer)
ProcCall(name: 'writeln', tab_index: 35, type: void)
Arguments:
  Var(name: 'y', tab_index: 39, type: integer)

```

==== TAB (Identifier Table) ====

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0

31 var	30 RESERVED NOTYP	0	1	0	0
32 while	31 RESERVED NOTYP	0	1	0	0
33 true	32 CONST BOOL	0	1	0	1
34 false	33 CONST BOOL	0	1	0	0
35 writeln	34 PROC NOTYP	0	1	0	0
36 readln	35 PROC NOTYP	0	1	0	0
37 Test1	36 PROGRAM NOTYP	0	1	0	0
38 x	37 VAR INT	0	1	0	0
39 y	38 VAR INT	0	1	0	1
40 r	39 VAR REAL	0	1	0	2

=== BTAB (Block Table) ===

Idx	Last	Lpar	Psize	Vsize
-----	------	------	-------	-------

0	40	0	0	3
1	0	0	0	0

[Semantic analysis completed successfully]

3.2. Test Case 2

Input
<pre> program Test2; var hasil: Integer; function Add(a: Integer; b: Integer): Integer; begin Add := a + b; end; begin hasil := Add(2, 3); end. </pre>
Output

DECORATED AST:

```

-----
ProgramNode(name: 'Test2', tab_index: 37)
Declarations:
  VarDecl(name: 'hasil', type: integer, tab_index: 38, level: 0)
  FunctionDecl(name: 'Add', return_type: void)
    Parameters:
      VarDecl(name: 'a', type: integer, tab_index: 40, level: 1, parameter)
      VarDecl(name: 'b', type: integer, tab_index: 41, level: 1, parameter)
    Block(block_index: 1, lev: 1)
      Assign('Add' := a+b, type: void)
        Target:
          Var(name: 'Add', tab_index: 39, type: integer)
        Value:
          BinOp(op: '+', type: integer)
            Left:
              Var(name: 'a', tab_index: 40, type: integer, level: 1)
            Right:
              Var(name: 'b', tab_index: 41, type: integer, level: 1)
    Block(block_index: 2, lev: 1)
      Assign('hasil' := ident1parent2comma3rparent, type: void)
        Target:
          Var(name: 'hasil', tab_index: 38, type: integer)
        Value:
          ProcCall(name: 'Add', tab_index: 39, type: void)
            Arguments:
              Number(int: 2, type: integer)
              Number(int: 3, type: integer)

```

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0

13 if	12 RESERVED NOTYP	0	1	0	0
14 mod	13 RESERVED NOTYP	0	1	0	0
15 not	14 RESERVED NOTYP	0	1	0	0
16 of	15 RESERVED NOTYP	0	1	0	0
17 or	16 RESERVED NOTYP	0	1	0	0
18 procedure	17 RESERVED NOTYP	0	1	0	0
19 program	18 RESERVED NOTYP	0	1	0	0
20 record	19 RESERVED NOTYP	0	1	0	0
21 repeat	20 RESERVED NOTYP	0	1	0	0
22 integer	21 TYPE INT	0	1	0	0
23 real	22 TYPE REAL	0	1	0	0
24 boolean	23 TYPE BOOL	0	1	0	0
25 char	24 TYPE CHAR	0	1	0	0
26 string	25 TYPE STR	0	1	0	0
27 then	26 RESERVED NOTYP	0	1	0	0
28 to	27 RESERVED NOTYP	0	1	0	0
29 type	28 RESERVED NOTYP	0	1	0	0
30 until	29 RESERVED NOTYP	0	1	0	0
31 var	30 RESERVED NOTYP	0	1	0	0
32 while	31 RESERVED NOTYP	0	1	0	0
33 true	32 CONST BOOL	0	1	0	1
34 false	33 CONST BOOL	0	1	0	0
35 writeln	34 PROC NOTYP	0	1	0	0
36 readln	35 PROC NOTYP	0	1	0	0
37 Test2	36 PROGRAM NOTYP	0	1	0	0
38 hasil	37 VAR INT	0	1	0	0
39 Add	38 FUNC INT	1	1	0	0
40 a	39 VAR INT	0	1	1	0
41 b	40 VAR INT	0	1	1	1

=== BTAB (Block Table) ===

Idx Last Lpar Psze Vsze

0	39	0	0 1
1	41	41	2 0
2	0	0	0 0

[Semantic analysis completed successfully]

3.3. Test Case 3

Input
<pre>program Test3; type Person == record nama: String; umur: Integer; end; var p: Person; begin p.nama := 'Naura'; p.umur := 18; end.</pre>
Output
DECORATED AST: <pre>----- ProgramNode(name: 'Test3', tab_index: 37) Declarations: TypeDecl(name: 'Person', type: record, tab_index: 40, block_index: 1) RecordType(type: record, block_index: 1) VarDecl(name: 'nama', type: string, tab_index: 38, level: 1) VarDecl(name: 'umur', type: integer, tab_index: 39, level: 1) VarDecl(name: 'p', type: record, tab_index: 41, level: 0) Block(block_index: 2, lev: 1) Assign('? := 'Naura', type: void) Target: Var(name: 'p', tab_index: 41, type: string) .field: nama Value: String(value: "Naura") Assign('? := 18, type: void) Target: Var(name: 'p', tab_index: 41, type: integer) .field: umur Value: Number(int: 18, type: integer) -----</pre>

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0
31	var	30	RESERVED	NOTYP	0	1	0	0
32	while	31	RESERVED	NOTYP	0	1	0	0
33	true	32	CONST	BOOL	0	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	writeln	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	0
37	Test3	36	PROGRAM	NOTYP	0	1	0	0
38	nama	37	VAR	STR	0	1	1	0
39	umur	38	VAR	INT	0	1	1	1
40	Person	37	TYPE	RECORD	1	1	0	0
41	p	40	VAR	RECORD	1	1	0	0

=== BTAB (Block Table) ===

Idx Last Lpar Psze Vsze

0 41 0 0 2
1 39 0 0 2
2 0 0 0 0

[Semantic analysis completed successfully]

3.4. Test Case 4

Input
<pre>program Test4; var x: integer; procedure increment; var x: integer; begin x := x + 1; end; begin x := 0; increment; writeln(x); { x global, masih 0 } end.</pre>
Output

DECORATED AST:

```

-----
ProgramNode(name: 'Test4', tab_index: 37)
Declarations:
  VarDecl(name: 'x', type: integer, tab_index: 38, level: 0)
  ProcedureDecl(name: 'increment')
    Declarations:
      VarDecl(name: 'x', type: integer, tab_index: 40, level: 1)
    Block(block_index: 1, lev: 1)
      Assign('x' := x+1, type: void)
        Target:
          Var(name: 'x', tab_index: 40, type: integer, level: 1)
        Value:
          BinOp(op: '+', type: integer)
            Left:
              Var(name: 'x', tab_index: 40, type: integer, level: 1)
            Right:
              Number(int: 1, type: integer)
    Block(block_index: 2, lev: 1)
      Assign('x' := 0, type: void)
        Target:
          Var(name: 'x', tab_index: 38, type: integer)
        Value:
          Number(int: 0, type: integer)
  ProcCall(name: 'increment', tab_index: 39, type: void)
  ProcCall(name: 'writeln', tab_index: 35, type: void)
    Arguments:
      Var(name: 'x', tab_index: 38, type: integer)

```

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0

13 if	12 RESERVED NOTYP	0	1	0	0
14 mod	13 RESERVED NOTYP	0	1	0	0
15 not	14 RESERVED NOTYP	0	1	0	0
16 of	15 RESERVED NOTYP	0	1	0	0
17 or	16 RESERVED NOTYP	0	1	0	0
18 procedure	17 RESERVED NOTYP	0	1	0	0
19 program	18 RESERVED NOTYP	0	1	0	0
20 record	19 RESERVED NOTYP	0	1	0	0
21 repeat	20 RESERVED NOTYP	0	1	0	0
22 integer	21 TYPE INT	0	1	0	0
23 real	22 TYPE REAL	0	1	0	0
24 boolean	23 TYPE BOOL	0	1	0	0
25 char	24 TYPE CHAR	0	1	0	0
26 string	25 TYPE STR	0	1	0	0
27 then	26 RESERVED NOTYP	0	1	0	0
28 to	27 RESERVED NOTYP	0	1	0	0
29 type	28 RESERVED NOTYP	0	1	0	0
30 until	29 RESERVED NOTYP	0	1	0	0
31 var	30 RESERVED NOTYP	0	1	0	0
32 while	31 RESERVED NOTYP	0	1	0	0
33 true	32 CONST BOOL	0	1	0	1
34 false	33 CONST BOOL	0	1	0	0
35 writeln	34 PROC NOTYP	0	1	0	0
36 readln	35 PROC NOTYP	0	1	0	0
37 Test4	36 PROGRAM NOTYP	0	1	0	0
38 x	37 VAR INT	0	1	0	0
39 increment	38 PROC NOTYP	1	1	0	0
40 x	39 VAR INT	0	1	1	0

=== BTAB (Block Table) ===

Idx	Last	Lpar	Psze	Vsze
-----	------	------	------	------

0	39	0	0	1
1	40	39	0	1
2	0	0	0	0

[Semantic analysis completed successfully]

3.5. Test Case 5

Input

```

program Test5;
var
  nums: array [1..5] of integer;
  i: integer;
begin
  i := 1;
  nums[i] := 42;
  writeln(nums[i]);
end.

```

Output

DECORATED AST:

```

-----
ProgramNode(name: 'Test5', tab_index: 37)
Declarations:
  VarDecl(name: 'nums', type: array, tab_index: 38, level: 0)
  VarDecl(name: 'i', type: integer, tab_index: 39, level: 0)
Block(block_index: 1, lev: 1)
  Assign('i' := 1, type: void)
    Target:
      Var(name: 'i', tab_index: 39, type: integer)
    Value:
      Number(int: 1, type: integer)
  Assign('?' := 42, type: void)
    Target:
      Var(name: 'nums', tab_index: 38, type: integer)
      [Index]
    Value:
      Number(int: 42, type: integer)
  ProcCall(name: 'writeln', tab_index: 35, type: void)
    Arguments:
      Var(name: 'nums', tab_index: 38, type: integer)
      [Index]

```

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0

1 and	0 RESERVED NOTYP	0	1	0	0
2 array	1 RESERVED NOTYP	0	1	0	0
3 begin	2 RESERVED NOTYP	0	1	0	0
4 case	3 RESERVED NOTYP	0	1	0	0
5 const	4 RESERVED NOTYP	0	1	0	0
6 div	5 RESERVED NOTYP	0	1	0	0
7 downto	6 RESERVED NOTYP	0	1	0	0
8 do	7 RESERVED NOTYP	0	1	0	0
9 else	8 RESERVED NOTYP	0	1	0	0
10 end	9 RESERVED NOTYP	0	1	0	0
11 for	10 RESERVED NOTYP	0	1	0	0
12 function	11 RESERVED NOTYP	0	1	0	0
13 if	12 RESERVED NOTYP	0	1	0	0
14 mod	13 RESERVED NOTYP	0	1	0	0
15 not	14 RESERVED NOTYP	0	1	0	0
16 of	15 RESERVED NOTYP	0	1	0	0
17 or	16 RESERVED NOTYP	0	1	0	0
18 procedure	17 RESERVED NOTYP	0	1	0	0
19 program	18 RESERVED NOTYP	0	1	0	0
20 record	19 RESERVED NOTYP	0	1	0	0
21 repeat	20 RESERVED NOTYP	0	1	0	0
22 integer	21 TYPE INT	0	1	0	0
23 real	22 TYPE REAL	0	1	0	0
24 boolean	23 TYPE BOOL	0	1	0	0
25 char	24 TYPE CHAR	0	1	0	0
26 string	25 TYPE STR	0	1	0	0
27 then	26 RESERVED NOTYP	0	1	0	0
28 to	27 RESERVED NOTYP	0	1	0	0
29 type	28 RESERVED NOTYP	0	1	0	0
30 until	29 RESERVED NOTYP	0	1	0	0
31 var	30 RESERVED NOTYP	0	1	0	0
32 while	31 RESERVED NOTYP	0	1	0	0
33 true	32 CONST BOOL	0	1	0	1
34 false	33 CONST BOOL	0	1	0	0
35 writeln	34 PROC NOTYP	0	1	0	0
36 readln	35 PROC NOTYP	0	1	0	0
37 Test5	36 PROGRAM NOTYP	0	1	0	0
38 nums	37 VAR ARRAY	1	1	0	0
39 i	38 VAR INT	0	1	0	5

=== BTAB (Block Table) ===

Idx Last Lpar Psze Vsze

0	39	0	0	6
1	0	0	0	0

=== ATAB (Array Table) ===

Idx	XTyp	EType	Eref	Low	High	Elsz	Size
-----	------	-------	------	-----	------	------	------

1	SUBRANGE	INT		0	1	5	1 5
---	----------	-----	--	---	---	---	-----

[Semantic analysis completed successfully]

3.6. Test Case 6 (Error Handling)

Menguji kondisi pada if-statement yang bukan bertipe Boolean. n bertipe Integer, tapi digunakan langsung sebagai kondisi if n then. Ini harus ditolak karena ekspresi di dalam if-statement wajib bertipe Boolean. Expected: fail.

Input

```
program Test6;
var
  n: integer;
begin
  n := 5;
  if n then      { n bertipe integer, bukan boolean }
    writeln('yes');
end.
```

Output

[SEMANTIC ERROR] Line 6: If condition must be Boolean, got integer

DECORATED AST:

```
-----
ProgramNode(name: 'Test6', tab_index: 37)
Declarations:
  VarDecl(name: 'n', type: integer, tab_index: 38, level: 0)
Block(block_index: 1, lev: 1)
  Assign('n' := 5, type: void)
    Target:
      Var(name: 'n', tab_index: 38, type: integer)
    Value:
      Number(int: 5, type: integer)
  If(type: void)
    Condition:
      Var(name: 'n', tab_index: 38, type: integer)
    Then:
      ProcCall(name: 'writeln', tab_index: 35, type: void)
        Arguments:
          String(value: "yes")
-----
```

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0

21 repeat	20 RESERVED NOTYP	0	1	0	0
22 integer	21 TYPE INT	0	1	0	0
23 real	22 TYPE REAL	0	1	0	0
24 boolean	23 TYPE BOOL	0	1	0	0
25 char	24 TYPE CHAR	0	1	0	0
26 string	25 TYPE STR	0	1	0	0
27 then	26 RESERVED NOTYP	0	1	0	0
28 to	27 RESERVED NOTYP	0	1	0	0
29 type	28 RESERVED NOTYP	0	1	0	0
30 until	29 RESERVED NOTYP	0	1	0	0
31 var	30 RESERVED NOTYP	0	1	0	0
32 while	31 RESERVED NOTYP	0	1	0	0
33 true	32 CONST BOOL	0	1	0	1
34 false	33 CONST BOOL	0	1	0	0
35 writeln	34 PROC NOTYP	0	1	0	0
36 readln	35 PROC NOTYP	0	1	0	0
37 Test6	36 PROGRAM NOTYP	0	1	0	0
38 n	37 VAR INT	0	1	0	0

=== BTAB (Block Table) ===

Idx Last Lpar Psze Vsze

0	38	0	0	1
1	0	0	0	0

[Semantic analysis completed with ERRORS]

3.7. Test Case 7 (Error Handling)

Menguji deklarasi ulang identifier dalam scope yang sama. score sudah dideklarasikan sebagai integer, lalu dideklarasikan lagi sebagai real dalam blok var yang sama. Ini harus ditolak karena satu identifier tidak boleh dideklarasikan lebih dari satu kali dalam scope yang sama. Expected: fail.

Input

```

program Test7;
var
  score: integer;
  score: real;    { redeclaration di scope yang sama }
begin
  score := 100;
end.

```

Output

[SEMANTIC ERROR] Identifier 'score' already declared in this scope

DECORATED AST:

```

-----
ProgramNode(name: 'Test7', tab_index: 37)
  Declarations:
    VarDecl(name: 'score', type: integer, tab_index: 38, level: 0)
    VarDecl(name: 'score', type: real, tab_index: 38, level: 0)
  Block(block_index: 1, lev: 1)
    Assign('score' := 100, type: void)
      Target:
        Var(name: 'score', tab_index: 38, type: integer)
      Value:
        Number(int: 100, type: integer)
-----

```

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0

9 else	8 RESERVED NOTYP	0	1	0	0
10 end	9 RESERVED NOTYP	0	1	0	0
11 for	10 RESERVED NOTYP	0	1	0	0
12 function	11 RESERVED NOTYP	0	1	0	0
13 if	12 RESERVED NOTYP	0	1	0	0
14 mod	13 RESERVED NOTYP	0	1	0	0
15 not	14 RESERVED NOTYP	0	1	0	0
16 of	15 RESERVED NOTYP	0	1	0	0
17 or	16 RESERVED NOTYP	0	1	0	0
18 procedure	17 RESERVED NOTYP	0	1	0	0
19 program	18 RESERVED NOTYP	0	1	0	0
20 record	19 RESERVED NOTYP	0	1	0	0
21 repeat	20 RESERVED NOTYP	0	1	0	0
22 integer	21 TYPE INT	0	1	0	0
23 real	22 TYPE REAL	0	1	0	0
24 boolean	23 TYPE BOOL	0	1	0	0
25 char	24 TYPE CHAR	0	1	0	0
26 string	25 TYPE STR	0	1	0	0
27 then	26 RESERVED NOTYP	0	1	0	0
28 to	27 RESERVED NOTYP	0	1	0	0
29 type	28 RESERVED NOTYP	0	1	0	0
30 until	29 RESERVED NOTYP	0	1	0	0
31 var	30 RESERVED NOTYP	0	1	0	0
32 while	31 RESERVED NOTYP	0	1	0	0
33 true	32 CONST BOOL	0	1	0	1
34 false	33 CONST BOOL	0	1	0	0
35 writeln	34 PROC NOTYP	0	1	0	0
36 readln	35 PROC NOTYP	0	1	0	0
37 Test7	36 PROGRAM NOTYP	0	1	0	0
38 score	37 VAR INT	0	1	0	0

=== BTAB (Block Table) ===

Idx Last Lpar Psze Vsize

0	38	0	0	2
1	0	0	0	0

[Semantic analysis completed with ERRORS]

3.8. Test Case 8 (Error Handling)

Menguji assignment-compatibility pada tipe subrange. index bertipe subrange 1..10, sehingga index := count valid karena nilai count (5) masuk dalam rentang, namun index

`:= 11` harus ditolak karena nilai 11 berada di luar batas atas subrange `[1..10]`. Expected: fail.

Input
<pre>program Test8; var index: 1..10; count: integer; begin count := 5; index := count; { OK: integer value 5 adalah subset dari [1..10] } index := 11; { ERROR: 11 di luar range [1..10] } end.</pre>
Output
[SEMANTIC ERROR] Line 8: Assignment-incompatible. Value 11 is out of range [1..10] for variable 'index' DECORATED AST: ----- ProgramNode(name: 'Test8', tab_index: 37) Declarations: - VarDecl(name: 'index', type: subrange, tab_index: 38, level: 0) - VarDecl(name: 'count', type: integer, tab_index: 39, level: 0) Block(block_index: 1, lev: 1) Assign('count' := 5, type: void) Target: - Var(name: 'count', tab_index: 39, type: integer) Value: - Number(int: 5, type: integer) Assign('index' := count, type: void) Target: - Var(name: 'index', tab_index: 38, type: subrange) Value: - Var(name: 'count', tab_index: 39, type: integer) Assign('index' := 11, type: void) Target: - Var(name: 'index', tab_index: 38, type: subrange) Value:

Number(int: 11, type: integer)

==== TAB (Identifier Table) ====

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0
31	var	30	RESERVED	NOTYP	0	1	0	0
32	while	31	RESERVED	NOTYP	0	1	0	0
33	true	32	CONST	BOOL	0	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	writeln	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	0
37	Test8	36	PROGRAM	NOTYP	0	1	0	0
38	index	37	VAR	SUBRANGE	1	1	0	0
39	count	38	VAR	INT	0	1	0	1

=== BTAB (Block Table) ===

Idx Last Lpar Psze Vsize

0 39 0 0 2
1 0 0 0 0

[Semantic analysis completed with ERRORS]

3.9. Test Case 9 (Error Handling)

Menguji assignment ke tipe yang tidak cocok. angka bertipe Integer, tapi diassign dengan literal string 'teks'. Ini harus ditolak karena assignment dari String ke Integer tidak kompatibel berdasarkan Type Compatibility Rule. Expected: fail.

Input
program Test9; var angka: Integer; begin angka := 'teks'; end.
Output

[SEMANTIC ERROR] Line 7: Type mismatch in assignment: cannot assign string to integer

DECORATED AST:

ProgramNode(name: 'Test9', tab_index: 37)

Declarations:

VarDecl(name: 'angka', type: integer, tab_index: 38, level: 0)

Block(block_index: 1, lev: 1)

Assign('angka' := 'teks', type: void)

Target:

Var(name: 'angka', tab_index: 38, type: integer)

Value:

String(value: "teks")

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0

28 to	27 RESERVED NOTYP	0	1	0	0
29 type	28 RESERVED NOTYP	0	1	0	0
30 until	29 RESERVED NOTYP	0	1	0	0
31 var	30 RESERVED NOTYP	0	1	0	0
32 while	31 RESERVED NOTYP	0	1	0	0
33 true	32 CONST BOOL	0	1	0	1
34 false	33 CONST BOOL	0	1	0	0
35 writeln	34 PROC NOTYP	0	1	0	0
36 readln	35 PROC NOTYP	0	1	0	0
37 Test9	36 PROGRAM NOTYP	0	1	0	0
38 angka	37 VAR INT	0	1	0	0

=== BTAB (Block Table) ===

Idx Last Lpar Psze Vsze

0	38	0	0	1
1	0	0	0	0

[Semantic analysis completed with ERRORS]

3.10. Test Case 10 (Error Handling)

Menguji akses field record yang tidak ada. p bertipe record Person yang hanya memiliki field nama dan umur, namun program mencoba mengakses p.alamat yang tidak dideklarasikan di dalam definisi record tersebut. Ini harus ditolak karena field 'alamat' tidak ditemukan pada record Person. Expected: fail.

Input

```

program Test10;

type
  Person == record
    nama: String;
    umur: Integer;
  end;

var
  p: Person;

begin
  p.nama := 'Budi';
  p.alamat := 'Bandung';
end.

```

Output

[SEMANTIC ERROR] Line 14: Undefined record field: alamat
 [SEMANTIC ERROR] Line 14: Type mismatch in assignment: cannot assign string to unknown

DECORATED AST:

```

-----
ProgramNode(name: 'Test10', tab_index: 37)
  Declarations:
    TypeDecl(name: 'Person', type: record, tab_index: 40, block_index: 1)
      RecordType(type: record, block_index: 1)
        VarDecl(name: 'nama', type: string, tab_index: 38, level: 1)
        VarDecl(name: 'umur', type: integer, tab_index: 39, level: 1)
        VarDecl(name: 'p', type: record, tab_index: 41, level: 0)
    Block(block_index: 2, lev: 1)
      Assign('? := 'Budi', type: void)
        Target:
          Var(name: 'p', tab_index: 41, type: string)
            .field: nama
        Value:
          String(value: "Budi")
      Assign('? := 'Bandung', type: void)
        Target:
          Var(name: 'p', tab_index: 41, type: record)
            .field: alamat
        Value:
          String(value: "Bandung")
-----

```

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0
31	var	30	RESERVED	NOTYP	0	1	0	0
32	while	31	RESERVED	NOTYP	0	1	0	0
33	true	32	CONST	BOOL	0	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	writeln	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	0
37	Test10	36	PROGRAM	NOTYP	0	1	0	0
38	nama	37	VAR	STR	0	1	1	0
39	umur	38	VAR	INT	0	1	1	1
40	Person	37	TYPE	RECORD	1	1	0	0
41	p	40	VAR	RECORD	1	1	0	0

=== BTAB (Block Table) ===

Idx Last Lpar Psze Vsze

0 41 0 0 2

1 39 0 0 2

2 0 0 0 0

[Semantic analysis completed with ERRORS]

BAB IV

Penutup

4.1. Kesimpulan

Berdasarkan hasil pengujian yang dilakukan pada Milestone 3 (Semantic Analysis), dapat disimpulkan bahwa:

- 1) Fungsi dasar semantic analysis telah berhasil diimplementasikan. Program mampu melakukan analisis semantik terhadap program Arion, meliputi:
 - Pembuatan dan pengelolaan tiga tabel simbol (tab, btab, atab) sesuai spesifikasi.
 - Traversal parse tree dengan fungsi visitor untuk melakukan deklarasi identifier, pengecekan tipe, dan anotasi AST.
 - Penerapan aturan kompatibilitas tipe dasar.
 - Pendeteksian error semantik.
- 2) Semua fitur wajib telah tercakup
 - Deklarasi konstanta, tipe, variabel, prosedur, dan fungsi dapat diproses.
 - Tipe data bentukan (array, record, enumerasi, subrange) ditangani dengan baik.
 - Ekspresi aritmatika, relasional, dan logika mendapatkan inferensi tipe yang benar.
 - Statement kontrol (if, while, repeat, for, case) diperiksa validitas tipenya.
 - Pemanggilan prosedur/fungsi dikenali, meskipun pengecekan parameter masih terbatas.
- 3) Pengujian berhasil
 - Dari 10 kasus uji, 5 kasus program yang benar secara semantik berhasil diproses tanpa error.
 - 5 kasus lainnya sengaja dirancang mengandung error semantik (misal tipe tidak cocok, identifier belum dideklarasikan, duplikasi nama), dan program berhasil mendeteksi serta melaporkan error tersebut tanpa crash.

4.2. Saran

- 1) Pengecekan parameter prosedur/fungsi
Saat ini pemanggilan writeln dan readln masih belum melewati pengecekan, dan secara umum jumlah serta tipe parameter belum divalidasi. Menambahkan pencocokan parameter akan membuat semantic analysis lebih ketat dan mendeteksi error pemanggilan fungsi yang salah.
- 2) Peringatan untuk variabel yang tidak digunakan
Menambahkan warning jika suatu variabel dideklarasikan tetapi tidak pernah digunakan (unused variable) dapat membantu untuk menulis kode yang lebih bersih.

3) Penanganan break/continue dalam loop

Meskipun tidak diwajibkan di spesifikasi, menambahkan validasi bahwa break dan continue hanya muncul di dalam loop dapat meningkatkan kualitas analisis.

4) Optimasi representasi AST

Saat ini parse tree dengan label tetap dipertahankan. Mengubahnya menjadi AST murni (menghilangkan node sintaksis seperti <expression>, <term>) akan memudahkan tahap code generation nanti. Namun hal ini memerlukan perubahan signifikan pada parser.

5) Dokumentasi kode

Beberapa fungsi visitor cukup panjang, menambahkan komentar singkat tentang aturan semantik yang diterapkan akan membantu anggota tim lain (atau asisten) memahami alur.

LAMPIRAN

Link Repository Github: <https://github.com/Zannn210/WOK-Tubes-IF2224-2026>

Tabel Pembagian Tugas

No	Tugas	Anggota	Persentase
1	- Mengimplementasikan visitor structured type: visitArrayType, visitRecordType, visitFieldList, dan visitFieldPart. - Menerapkan error handling menyeluruh melalui fungsi semanticError dengan nomor baris, serta validasi tipe pada setiap node ekspresi. - Menyiapkan minimal 5 test case unik dan memastikan Decorated AST serta Symbol Table tercetak ke terminal dan file keluaran.	Fauzan Mohamad Abdul Ghani (13524113)	20%
2	- Mengimplementasikan 3 Symbol Table utama: tab, btab, atab. - Mendaftarkan Predefined Identifier melalui initPredefined(): tipe Integer, Real, Char, Boolean, String, serta konstanta true/false dan prosedur writeln/readln. - Membuat fungsi manajemen scope: enterBlock, leaveBlock, addIdentifier, dan lookupIdent. - Mengimplementasikan visitor deklarasi: visitConstDeclaration, visitConstant, visitTypeDeclaration, visitType, dan visitVarDeclaration.	Safira Berlianti (13524128)	20%
3	- Mengimplementasikan visitor subprogram: visitSubprogramDeclaration, visitProcedureDeclaration, visitFunctionDeclaration, visitFormalParameterList, dan visitParameterGroup. - Mengimplementasikan visitor akses variabel: visitVariable, visitComponentVariable, dan visitIndexList. - Mengimplementasikan visitor visitProcFuncCall dan visitAssignmentStatement beserta validasi assignment-compatibility dan bounds checking untuk subrange.	Eduard Daniel Ariajaya (13524129)	20%
4	- Mengimplementasikan visitor visitStatementList dan visitStatement sebagai dispatcher ke semua jenis statement. - Mengimplementasikan visitor percabangan: visitIfStatement, visitCaseStatement, dan visitCaseBlock.	Muhammad Daffa Arrizki Yanma (13524133)	20%

	• Mengimplementasikan visitor perulangan: visitWhileStatement, visitRepeatStatement, dan visitForStatement.		
5	• Menambahkan field anotasi semantik pada ASTNode: semType, tabIdx, lexLevel, dan line untuk menyimpan hasil analisis tipe, referensi symbol table, dan nomor baris. • Mengimplementasikan ASTDecoratedPrinter untuk mencetak Decorated AST beserta informasi tipe dan isi symbol table ke terminal dan file keluaran. • Mengimplementasikan visitor untuk kerangka utama program: visitProgram, visitDeclarationPart, visitBlock, visitCompoundStatement, visitStatementList, dan visitStatement sebagai dispatcher ke visitor lainnya.	Reysya Syafitri Mulya Ramadhan (13524137)	20%

REFERENSI

<https://docs.google.com/document/d/1VhU46bYf3ciP-EOtAiZ9XlHHVc-tG4h1BHf-NoGOoRw/edit?tab=t.0>

<https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>

<https://ruslanspivak.com/lbasi-part7/>

<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>